

SCHEDULING.

Document your implementation, including design choices, code structure, and performance analysis.

I followed the template given by the teacher. It was really easy to follow those steps and there wasn't really anything to design, the only thing that i changed about the template is that i changed the position of the sleep→.

```
if (execution_time > TRAJECTORY_PERIOD_MS * 1000) {  
    printf("Trajectory task missed  
deadline!*****\n");  
} else { usleep((MOTOR_PERIOD_MS * 1000) - execution_time);}  
// Sleep for task period if i sleep when deadline is missed i will get into a bottleneck and  
stop the system
```

The code implemented:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <unistd.h>  
#include <sched.h>  
#include <time.h>  
  
// Function to generate random sensor data  
int generateSensorData() {  
    return rand() % 61; // Random number between 0 and 60  
}  
  
// Define struct for sensor and motor data  
struct SensorData {  
    // Sensor data  
    int e1;  
    int e2;  
    int e3;  
    int e4;  
    int e5;  
    // Motor data
```

```

int M1;
int M2;
int M3;
int M4;
int M5;
};

// Define task periods in microseconds
#define SENSOR_PERIOD_MS 50
#define TRAJECTORY_PERIOD_MS 100
#define MOTOR_PERIOD_MS 10

// Mutexes for thread synchronization
pthread_mutex_t sensorMutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t trajectoryMutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t motorMutex = PTHREAD_MUTEX_INITIALIZER;
void print_motor(void*arg){
    // Cast the void pointer to the struct type
    struct SensorData *M = (struct SensorData *)arg;
    printf("Moving to trajectory computed
[%i,%i,%i,%i,%i]...\n",M->M1,M->M2,M->M3,M->M4,M->M5);
}

// Function to print the current time
void print_current_time() {
    // TODO: Implement this function to print the current time in a human-readable format
    // calendar time
    time_t current_time;
    struct tm *local_time;
    char time_str[100];

    // Get the current time
    time(&current_time);

    // Convert the current time to the local time
    local_time = localtime(&current_time);

```

```

    // Format the local time into a human-readable string
    strftime(time_str, sizeof(time_str), "%a, %d %b %Y %H:%M:%S %Z", local_time);

    // Print the formatted time
    printf("Current time: %s\n", time_str);
}

// Motor control task function
void* motor_task(void* arg) {
    struct timespec start, end;
    // Cast the void pointer to the struct type
    struct SensorData *M = (struct SensorData *)arg;
    // TODO: Implement the motor control task
    while (1) {
        clock_gettime(CLOCK_MONOTONIC, &start);
        // Acquire motor control mutex
        pthread_mutex_lock(&motorMutex);
        print_motor(M);
        // Print current time
        print_current_time();

        // Release motor control mutex
        pthread_mutex_unlock(&motorMutex);
        clock_gettime(CLOCK_MONOTONIC, &end);

        // Sleep for task period
        long execution_time = (end.tv_sec - start.tv_sec) * 1000000 + (end.tv_nsec -
start.tv_nsec) / 1000;

        if (execution_time > MOTOR_PERIOD_MS * 1000) {
            printf("Motor task missed deadline!*****\n");
        }
        else
        { usleep((MOTOR_PERIOD_MS * 1000) - execution_time);}
    }
}

```

```

    }
    return NULL;
}

// Sensor data processing task function
void* sensor_task(void* arg) {
    struct timespec start, end;
    // Cast the void pointer to the struct type
    struct SensorData *M = (struct SensorData *)arg;
    // TODO: Implement the sensor data processing task
    while (1) {
        clock_gettime(CLOCK_MONOTONIC, &start);
        // Acquire sensor data mutex
        pthread_mutex_lock(&sensorMutex);
        M->e1 = generateSensorData();
        M->e2 = generateSensorData();
        M->e3 = generateSensorData();
        M->e4 = generateSensorData();
        M->e5 = generateSensorData();
        printf("Sensor readed: [%i,%i,%i,%i,%i]...\n",M->e1,M->e2,M->e3,M->e4,M->e5);
        // Print current time
        print_current_time();

        // Release sensor data mutex
        pthread_mutex_unlock(&sensorMutex);
        clock_gettime(CLOCK_MONOTONIC, &end);
        long execution_time = (end.tv_sec - start.tv_sec) * 1000000 + (end.tv_nsec -
start.tv_nsec) / 1000;

        if (execution_time > SENSOR_PERIOD_MS * 1000) {
            printf("Sensor task missed deadline!*****\n");
        } else { usleep((MOTOR_PERIOD_MS * 1000) - execution_time);}
        // Sleep for task period

    }
    return NULL;
}

```

```

// Trajectory planning task function
void* trajectory_task(void* arg) {
    struct timespec start, end;
    // Cast the void pointer to the struct type
    struct SensorData *M = (struct SensorData *)arg;
    // TODO: Implement the trajectory planning task
    while (1) {
        clock_gettime(CLOCK_MONOTONIC, &start);
        // Acquire trajectory planning mutex
        pthread_mutex_lock(&trajectoryMutex);
        M->M1 = M->e1 - M->e5;
        M->M2 = M->e1 + 2 * M->e2;
        M->M3 = M->e2 + M->e3;
        M->M4 = M->e4 + 4 * M->e3;
        M->M5 = M->e4 - 2 * M->e5;

        printf("Calculating trajectory...\n");
        // Print current time
        print_current_time();
        // Release trajectory planning mutex
        pthread_mutex_unlock(&trajectoryMutex);
        clock_gettime(CLOCK_MONOTONIC, &end);
        long execution_time = (end.tv_sec - start.tv_sec) * 1000000 + (end.tv_nsec -
start.tv_nsec) / 1000;

        if (execution_time > TRAJECTORY_PERIOD_MS * 1000) {
            printf("Trajectory task missed
deadline!*****\n");
        } else { usleep((MOTOR_PERIOD_MS * 1000) - execution_time);}
        // Sleep for task period if i sleep when deadline is missed i will get into a bottleneck

    }
    return NULL;
}

```

```
int main() {
    srand(time(NULL));

    struct SensorData M;

    // Threads for each task
    pthread_t motor_thread, sensor_thread, trajectory_thread;
    pthread_attr_t attr;

    // Set scheduling policy to SCHED_FIFO for real-time behavior
    struct sched_param param;
    pthread_attr_init(&attr);
    pthread_attr_setschedpolicy(&attr, SCHED_FIFO);

    // Adjust priority as needed

    // Create motor control thread
    param.sched_priority = 40; // Example priority
    pthread_attr_setschedparam(&attr, &param);
    pthread_create(&motor_thread, &attr, motor_task, &M);

    // Create sensor data processing thread
    param.sched_priority = 30; // Example priority
    pthread_attr_setschedparam(&attr, &param);
    pthread_create(&sensor_thread, &attr, sensor_task, &M);

    // Create trajectory planning thread
    param.sched_priority = 20; // Example priority
    pthread_attr_setschedparam(&attr, &param);
    pthread_create(&trajectory_thread, &attr, trajectory_task, &M);

    // Main thread acts as the real-time task scheduler

    // Wait for threads to finish (should never happen) and join them
    pthread_join(sensor_thread, NULL);
    pthread_join(trajectory_thread, NULL);
```

```
pthread_join(motor_thread, NULL);

return 0;
}
```

Test of system under distinct loads.

Case1:

The system has plenty of time to compute all calculations

```
// Define task periods in microseconds
#define SENSOR_PERIOD_MS 50
#define TRAJECTORY_PERIOD_MS 100
#define MOTOR_PERIOD_MS 10
```

```
aulavirtual@Linuxv3047:~/lab5$ ./lab5
Moving to trajectory computed [0,0,0,0,0]...
Sensor readed: [28,18,57,13,3]...
Calculating trajectory...
Current time: Tue, 14 May 2024 12:56:47 CEST
Current time: Tue, 14 May 2024 12:56:47 CEST
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Sensor readed: [38,16,31,9,54]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Moving to trajectory computed [25,64,75,241,7]...
Current time: Tue, 14 May 2024 12:56:47 CEST
Sensor readed: [5,18,5,58,5]...
Calculating trajectory...
```

Case2:

Around the time specified below the system starts to fail to complete all tasks and give its resources to the highest priority task, the motor, the response is alternating between success and fail of the completion of the motor task.

```
// Define task periods in microseconds
#define SENSOR_PERIOD_MS 0.25
#define TRAJECTORY_PERIOD_MS 0.5|
#define MOTOR_PERIOD_MS 0.05
```

```
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Motor task missed deadline!*****
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Motor task missed deadline!*****
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Motor task missed deadline!*****
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Motor task missed deadline!*****
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Motor task missed deadline!*****
Moving to trajectory computed [-49,56,79,277,-57]...
Current time: Tue, 14 May 2024 13:10:10 CEST
Moving to trajectory computed [-49,56,79,277,-57]...
```

Case3:

Around this time the system fails miserably, we can observe that it rarely succeed in beating the time defined

```
// Define task periods in microseconds
#define SENSOR_PERIOD_MS 0.05
#define TRAJECTORY_PERIOD_MS 0.1
#define MOTOR_PERIOD_MS 0.01
```

[illegible]

The performance overall is correct but we have to be careful with the deadlines, once the systems starts to fail to complete the highest priority task will ignore the other tasks, in this example we can see that it tries to keep up with the deadline but since it is never reading from the sensor, the final path doesn't change and the motos output remains unchanged.